

PIC

Verification Specification

Reproducible stimuli, checks and waveform evidence

Document	PIC — Verification Specification
Document type	Verification specification
Subject	Reproducible stimuli, checks and waveform evidence
Revision	2.2
Date	23 September 2026
Author	Bunea Cosmin-Andrei
Status	Released for review

Contents

1	Introduction	2
1.1	Purpose of the document	2
1.2	Validity of the document	2
1.3	Definitions of terms and abbreviations	2
1.4	Relationship with other documents	2
1.5	Notes on the notation used	3
2	Verification methodology	4
2.1	Overview	4
2.2	Implementation strategy	4
2.3	Verification plans	4
2.4	Reference model	5
3	Verification environment	6
3.1	Overview	6
3.2	Control and drivers	6
3.2.1	Clock and reset generator	7
3.2.2	AXI4-Lite master tasks	7
3.2.3	Source and CPU-side drivers	7
3.2.4	Random traffic	7
3.3	Monitors and checkers	7
3.3.1	AXI protocol monitor	7
3.3.2	Assertions	8
3.3.3	Result checkers	8
3.3.4	Formal harness	8
3.3.5	Non-vacuous checks	8
3.4	Simulation environment setup	8
3.4.1	Structure	8
3.4.2	Scripts	9
3.4.3	Setup and configuration	9
4	Annex	10
4.1	Verification plan	10
4.2	Critical sequences	11
4.3	External AXI reference	12

1 Introduction

1.1 Purpose of the document

This specification describes the simulation strategy, stimulus, reference models and acceptance checks of the programmable interrupt controller. It is intended for hardware designers, verification engineers and software developers integrating the block.

It defines the behaviour that the RTL must show at its interfaces.

1.2 Validity of the document

The documented configuration is the RTL in `cpu/hdl` and the associated CPU/PIC and SoC simulation environments. Interface adaptations from the supplied briefs are identified in the integration section.

Revision 2.2 of 23 September 2026 describes the delivered state of that RTL.

1.3 Definitions of terms and abbreviations

Term	Meaning
AXI4-Lite	Single-beat AXI read/write channels.
CSR	Control and status register.
DUT	Device under test.
EOI	End of interrupt; completes one service level.
IRQ	Interrupt request.
NBA	Nonblocking-assignment update region in simulation.
PIC	Programmable interrupt controller.
SVA	SystemVerilog Assertions.

1.4 Relationship with other documents

The documents below are the ones this specification is derived from or read with.

Document	Version / date	Author	Relationship
Programmable Interrupt Controller with Advanced Scheduling	Draft, 26 June 2026	D. Borzasi, Siemens	Functional requirements; see section 2.
Spec table of contents and templates	2 September 2026	D. Borzasi, Siemens	Document structure and verification-plan template.
Design Specification — PIC	Rev. 2.2, 23 September 2026	C.-A. Bunea	Companion document.
AMBA AXI and ACE Protocol Specification	IHI0022E	Arm	AXI4-Lite ports.
IEEE Std 1364 (Verilog)	2005	IEEE	RTL language.

1.5 Notes on the notation used

Signal and register names follow the RTL. Addresses and waveform bus values are hexadecimal; time is in ns and widths are in bits. Register offsets are byte offsets; CSR addresses are instruction-encoded indices.

Notation	Meaning
Fixed-width name	A Verilog module, port, file or script name, spelled as in the source.
Signal suffix <code>_i</code> / <code>_o</code>	Module input / output, seen from inside that module.
<code>[a:b]</code>	Bit range, most significant bit first.
Source slot	One of the 16 arbitration identities; x or n denotes its index 0–15.
<code>posedge +1 ns</code>	A testbench stimulus time, one nanosecond after the sampling edge.

2 Verification methodology

2.1 Overview

A run either passes or fails; this section states what that verdict covers. The first table gives the acceptance conditions applied to every test and the limits of what simulation shows; the second gives the runs behind the results quoted in this document.

Item	Contract
Requirement source	Programmable Interrupt Controller with Advanced Scheduling (original PIC brief).
Device under test	pic and axi_lite_slave.
Functional verdict	Every expected value and response must match; errors=0 and test_done=1 are required.
Protocol verdict	VALID/payload stability, response ordering and outstanding transaction limits; clocked at posedge.
Failure handling	Mismatch, timeout, incomplete test, assertion failure or coverage-gate failure returns a failing run.
Limits of evidence	Simulation against independent models, plus formal proofs of the invariants listed in PIC-10; no synthesis or timing result.

Simulator / flow	Recorded evidence (23 September 2026)
ModelSim	39 CPU/PIC runs; zero mismatches.
Verilator -assert	17 CPU/PIC benches in 53 runs; merged functional coverage 90/90 bins and every cover property reached.
Integrated SoC	34 ModelSim runs; 23 assertion-enabled benches in 51 Verilator runs.
SymbiYosys	Ten PIC invariants proven; each of five injected defects fails the proof.
Mutation check	25 injected RTL defects, each detected by its bench.
PULP comparison	Decoder over ten random seeds plus 96 register-slave strobe/order/stall cases and RO/unmapped/reset cases.

2.2 Implementation strategy

Directed tests exercise individual features and simultaneous events; program and reference tests check end-to-end results. Protocol checks observe accepted transfers; state checks sample after nonblocking updates.

Seeded random traffic and formal proofs extend the directed set. Every bench also runs under Verilator with the assertions bound.

2.3 Verification plans

The plan in section 4.1 follows the supplied template: ID, title, description, status and test/checker, one row per test.

2.4 Reference model

`pic_reference.py` computes the winner and pending mask of 275 static cases from the priority rule. `pic_model.py` is a cycle model of the controller, written from this specification; it replays the inputs recorded by `pic_tb_random` and must reproduce outputs and state on every cycle. `pic_formal.sv` states invariants that SymbiYosys proves for every input sequence.

3 Verification environment

3.1 Overview

Register-access tasks configure the PIC and drive source, mask and claim/EOI events. The independent model supplies expected winners and pending masks; the checkers also inspect register responses and service-state transitions.

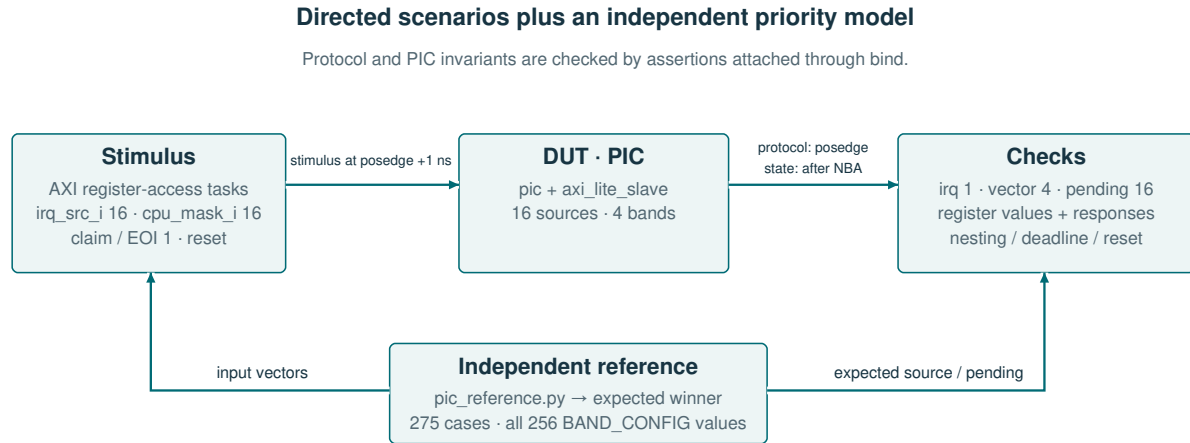


Figure 1: Test stimuli, DUT and independent checking paths.

Component	File / role
Clock / reset	ck_rst_tb.v: shared generator; dedicated reset benches can drive a reset pulse between edges.
Bus stimulus	tb_axil_master.vh: register-access tasks; CPU programs supply real load/store requests at system level.
Memory model	axi_lite_mem_model.v: configurable response delay and repeatable backpressure.
Reference	pic_reference.py: expected priority winner and pending mask.
Verdict	tb_check.vh: explicit mismatch count, completion and fatal failure.
Protocol	axi_lite_monitor.v for ModelSim; axi_lite_sva.sv under Verilator.

3.2 Control and drivers

Stimulus is driven one nanosecond after the sampling edge, so that a test never races the design it is checking. The table gives the timing rule for each driven event and the reason for it; the waveform shows the reset case, the one event placed off the clock grid.

Signal / event	Timing rule	Reason
clk	10 ns period in these traces; rising edges are 5, 15, 25... ns.	Defines the handshake observation points.
Shared bus / claim tasks	Drive 1 ns after posedge, in a separate statement. Hold until the sampling edge.	Avoid blocking-assignment races with DUT active/NBA evaluation.

Signal / event	Timing rule	Reason
Handshake observation	Sample VALID && READY at posedge before changing stimulus.	Records the transfer that the DUT accepted.
Reset assertion	Dedicated reset tests assert at posedge +2 ns, check at +3 ns.	The check occurs before either the next falling or rising edge.
Reset release	Explicit nanosecond delays, off the clock grid; shared generator releases at 123 ns.	Exercises asynchronous stimulus without a coincident clock event.

The controller starts with active nested service and enabled sources. Reset clears both depth and INT_ENABLE between clock edges; the test checks the state one nanosecond after assertion.

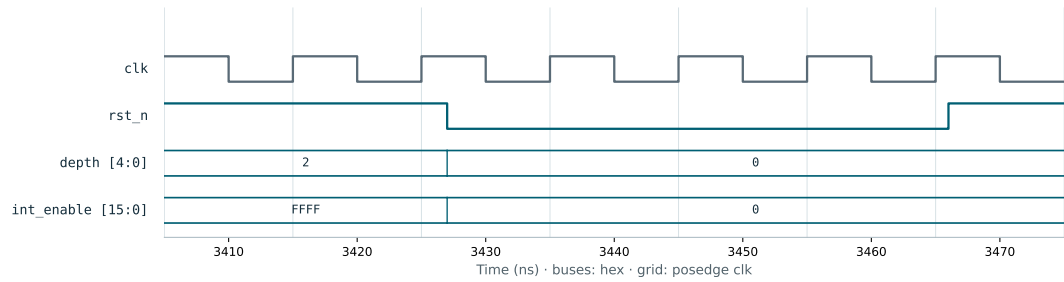


Figure 2: Reset from active state at posedge +2 ns; cleared state is checked at +3 ns.

3.2.1 Clock and reset generator

ck_rst_tb.v generates a 10 ns clock and holds reset from time 0 to 123 ns, off the clock edge. pic_tb_reset drives its own reset pulses between edges.

3.2.2 AXI4-Lite master tasks

tb_axil_master.vh provides register read and write tasks that check the response code. AW and W are issued together and may complete in either order.

3.2.3 Source and CPU-side drivers

The block benches drive irq_src_i, cpu_mask_i and the claim/EOI pair directly, in place of the CPU.

3.2.4 Random traffic

pic_tb_random draws register writes, sources, masks, claims and EOIs from a seed (+seed, +cycles) and writes every cycle to pic_random.trace.

3.3 Monitors and checkers

Checking is split between bus protocol and functional result, so that a failure points at one of the two.

3.3.1 AXI protocol monitor

axi_lite_monitor.v checks every transfer on the monitored ports in ModelSim and counts transactions; a run without traffic fails.

3.3.2 Assertions

`axi_lite_sva.sv` checks the register port; `pic_sva.sv` checks offer eligibility, the CPU mask, the priority threshold, depth transitions, claim/EOI exclusion and the empty EOI. `rv32i_bind_core_sva.sv` attaches the checkers to the RTL module types with `bind`; they only read DUT signals. Verilator runs them with `-assert`; ModelSim ASE runs the procedural monitors.

3.3.3 Result checkers

`tb_check.vh` counts mismatches and ends each run with an explicit verdict. `pic_tb_reference` compares winner and pending mask with `pic_reference.py`; `pic_model.py` replays each random trace and compares every cycle.

3.3.4 Formal harness

`pic_formal.sv` states the PIC invariants for SymbiYosys. `run_formal.sh` runs the proofs, then repeats them with five injected defects, each of which must fail.

3.3.5 Non-vacuous checks

Non-vacuous check	Evidence
Valid traffic	Read/write counts and exercised tests prevent a disconnected monitor from being counted as coverage.
Preconditions	RO-write test first creates nonzero state; async-reset test first creates pending/nonzero state.
Negative scenarios	Watch an interval for forbidden offers/claims; do not infer absence from one sample.

3.4 Simulation environment setup

The environment is a fixed folder layout plus a small number of named scripts. The tables give that layout, the exact command for each step in the order it is run, and the tools each step needs.

3.4.1 Structure

Folder	Contents
<code>cpu/hdl</code>	Synthesizable RTL: the CPU modules, the PIC, the timer and the AXI4-Lite register front-end.
<code>cpu/debug/hdl</code>	Testbenches, bus masters and the configurable memory and responder models.
<code>cpu/debug/sva</code>	Assertion modules and the bind files that attach them to RTL module types.
<code>cpu/debug/sim</code>	Run scripts, file lists, reference models, generated programs and run logs.
<code>soc/hdl</code>	Integration fabric: address decode, arbitration, AXI4-Full to Lite conversion and RAM.
<code>soc/debug/hdl</code> , <code>sva</code> , <code>sim</code>	The same split as above, applied to the integrated SoC.
<code>cpu/docs</code>	Specification sources, generated figures and the waveform windows.

3.4.2 Scripts

Step	Working directory	Command / script	Result
1	Repository root	make asm	Regenerate programs and independent reference vectors.
2	cpu/debug/sim	vsim -c -do "do regress.do; quit -f"	Compile and run 39 CPU/PIC runs.
3	cpu/debug/sim (Bash / WSL)	bash run_verilator.sh	Lint, 17 benches in 53 runs with SVA, coverage and cover gates.
4	soc/debug/sim	vsim -c -do "do regress.do; quit -f"	34 integration runs.
5	Repository root (Bash / WSL)	bash soc/debug/sim/run_verilator.sh	SoC lint and 23 assertion-enabled benches in 51 runs.
6	soc/debug/sim (Bash / WSL)	bash run_pulp_compare.sh	Pinned external decoder/register comparison.
7	Repository root (Bash / WSL)	bash cpu/debug/formal/run_formal.sh	PIC proofs, then the injected defects.
8	Repository root	python cpu/debug/sim/mutation_check.py	25 injected RTL defects.
9	cpu/debug/sim	vsim -c -do "do waves.do"	Run 8 checked benches and export VCD traces.
10	Repository root	python cpu/docs/tools/waveforms.py	SVG/PDF/PNG figures and source-window manifest.

3.4.3 Setup and configuration

Tool / setup	Requirement
ModelSim	vsim/vlog on PATH; run from the named simulation directory so file lists resolve.
Verilator	Verilator 5.050 used for the recorded run; C++ build tools, Bash and Python 3 are required.
PULP	Git and network only for the initial checkout; explicit revision pins recorded in the runner.
SymbiYosys	yosys, sby, yosys-abc and z3 on PATH.
Wave rendering	Python with matplotlib; raw VCD files produced by waves.do.
Configuration	Latency/stall overrides are read back after elaboration; seeds are fixed for repeatability.

4 Annex

4.1 Verification plan

PASS indicates the recorded regression completed with its expected results and zero checker errors. Each row identifies the test that exercises the listed behavior.

ID	Title	Description	Status	Test / checker
PIC-01	Feature sequences	Level/edge sources, priority bands/intra-band/ties, nesting, limit, spurious, escalation, SW trigger and access response codes.	PASS	pic_tb_feature
PIC-02	Priority reference	Independent priority model: 275 cases covering all 256 BAND_CONFIG encodings, request/enable/CPU masks and ties.	PASS	pic_tb_reference
PIC-03	Simultaneous events	Masked source cannot starve another; fresh edge coincident with claim; reordered-band escalation; trigger-key strobes.	PASS	pic_tb_sched
PIC-04	Asynchronous reset	All per-source/global defaults, loaded-state reset before next clock edge, reset with AW-only/W-only/B-held/R-held, recovery.	PASS	pic_tb_reset
PIC-05	Read-only registers	Write every RO register from a nonzero hardware state; expect SLVERR and unchanged value.	PASS	pic_tb_ro
PIC-06	Status transitions	PEND, ACTIVE, ESC, SPUR, EFF_BAND and timer fields; exact escalation transitions and complete source life cycle.	PASS	pic_tb_status
PIC-07	CPU integration	CPU programs the real PIC; claim/EOI order, masks, pending and interrupt handling across LSU stalls.	PASS	rv32i_tb_cpu_axi
PIC-08	SoC integration	Interrupt source routing, nesting to sixteen levels, escalation and a spurious claim with the real CPU in the integrated SoC.	PASS	soc_tb_pic_sources / nest / escalate / spurious / deep_nest
PIC-09	Random traffic	Random register writes, sources, masks, claims and EOIs, reconfiguration while nested included; every cycle equal to the cycle model pic_model.py.	PASS	pic_tb_random

ID	Title	Description	Status	Test / checker
PIC-10	Formal proofs	Proven for every input: stack depth at most 16, NEST_MAX in 1..16, a claim at the limit and an EOI on an empty stack change nothing, software requests only through a keyed write, only eligible sources offered; five injected defects each fail the proof.	PASS	pic_formal.sv (SymbiYosys)

4.2 Critical sequences

A second source edge is sampled with the claim of the first event. The edge latch remains set, and the source is offered again after EOI.

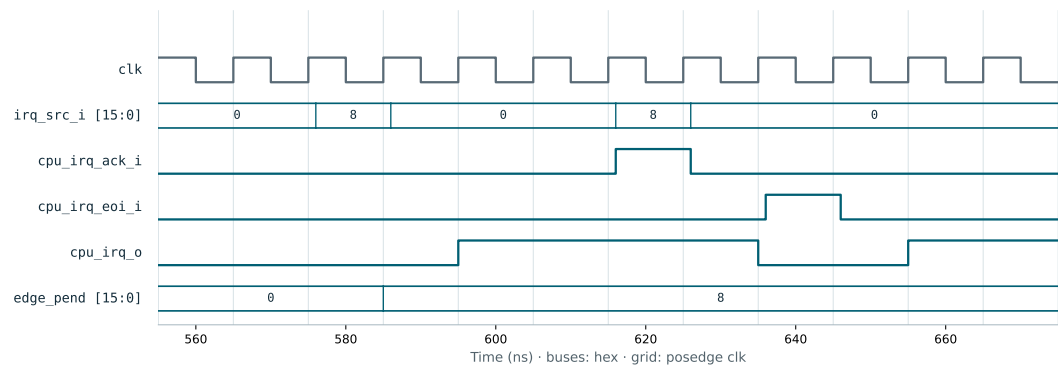


Figure 3: New edge coincident with claim: the edge latch remains set.

Source 1 initially wins the tie. Source 2 reaches its ten-cycle deadline; the escalation event changes its effective priority, and the registered offer changes to source 2 on the following edge.

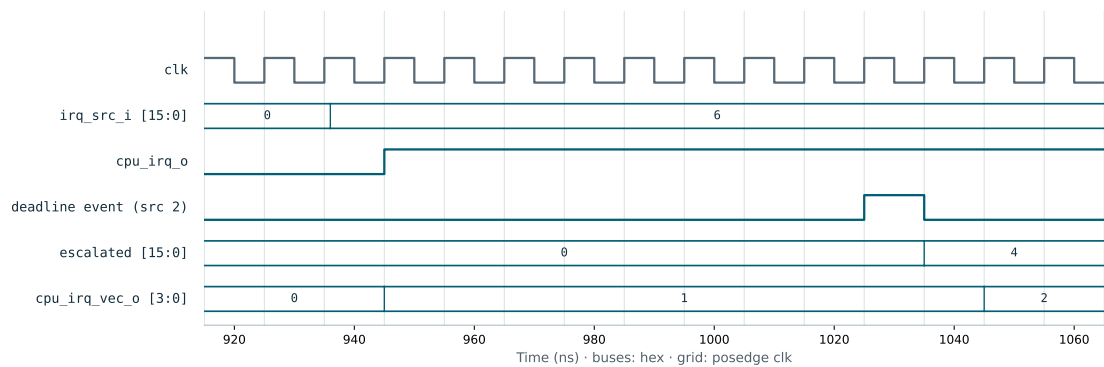


Figure 4: Ten-cycle deadline: escalation event, then registered source 2 offer under reordered band urgency.

Scenario	Check
New edge + claim	Old event is consumed, new event is retained.
Band reordering	Escalation follows programmed urgency, not numeric band order.
Nest limit + eligible source	Offer suppressed and overflow logged; existing stack is preserved.

4.3 External AXI reference

The AXI4-Lite front-end is compared against a published implementation to check the interpretation of the protocol. The first table states what was compared and on what evidence; the second names every difference found and what it means.

Compared block	Shared contract	Evidence
soc_axi_lite_dec vs axi_lite_demux	Two mapped targets, MaxTrans=1; independent progress and response timing.	Compare ordered data/response records and per-target address counts.
axi_lite_slave vs axi_lite_regs	32-bit words; four RW words and one RO word; same byte strobes.	96 cases = 16 WSTRB masks $\times$ 3 AW/W orders $\times$ 2 response-stall settings.
Additional register cases	RO write, unmapped access, reset and post-reset access.	Same access-class response and no forbidden state update; each side checked against expected data.

Difference / boundary	Interpretation
Latency	Not cycle-identical; compare completed transactions and protocol compliance.
Unmapped read data	Our front-end returns zero; pinned PULP axi_lite_regs returns 0xBA5E1E55. Both return SLVERR; error-data policy is documented separately.
Address aliases	Different register-bank sizing; equality claimed only for the explicit compared address profile.
AXI4-Full	The PULP run does not establish equivalence of the DMA or soc_axi_full2lite to every upstream AXI4 module.
Converter	Local soc_tb_full2lite and soc_tb_full2lite_err check supported burst handling and errors. Local write error aggregation preserves DECERR over SLVERR; PULP is not a cycle-exact reference for that policy.
Versions	axi: 70b8e54fd460e3308e58be596ceb3566a6e3576e; common_cells: 03d98106aa19952a10360d2230def85144a0008b.

Reference: <https://github.com/pulp-platform/axi>.